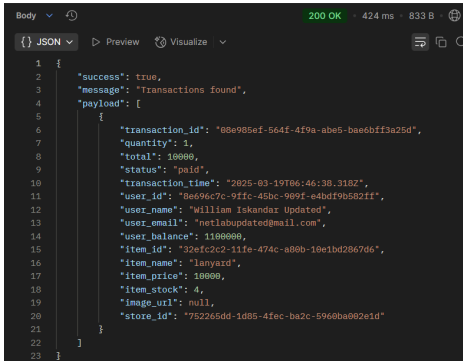
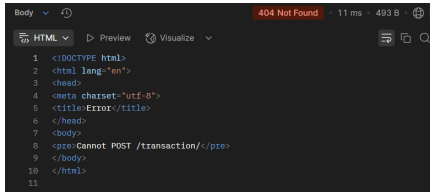


PRAKTIKUM SISTEM BASIS DATA

Nama	Jonathan Frederick Kosasih	No. Modul	6
NPM	2306225981	Tipe	TUTAM

1. Saya menambahkan fungsi getAllTransactions()
2. Screenshot:

Method	Endpoint	Success	Failed
GET	/transaction		

3. Penjelasan fitur untuk meningkatkan scalability, security, dan error handling:

Tipe	Implementasi	Deskripsi
Scalability	<pre>const emailRegex = /^[^s@]+@[^\s@]+\.[^\s@]+\$/; exports.registerUser = async (req, res) => { if (!req.query.name !req.query.email !req.query.password) { return baseResponse(res, false, 400, "Missing user name, email, or password", null); } try { if (!emailRegex.test(req.query.email)) { return baseResponse(res, false, 400, "Invalid email", null); } if (await userRepository.getUserByEmail(req.query.email)) { return baseResponse(res, false, 400, "Email already registered", null); } if (!passRegex.test(req.query.password)) { return baseResponse(res, false, 400, "Password must be at least 8 characters long, contain at least one number, and one special character", null); } const hashedPassword = await bcrypt.hash(req.query.password, saltRounds); const user = await userRepository.registerUser(req.query.name, req.query.email, hashedPassword); baseResponse(res, true, 201, "User created", user); } catch (error) { baseResponse(res, false, 500, error.message "Server error", error); } }</pre>	Pengecekan email menggunakan emailRegex saat register user dan update user, dikarenakan kemungkinan terjadinya penggunaan email dengan bentuk yang tidak valid untuk memudahkan pengolahan data

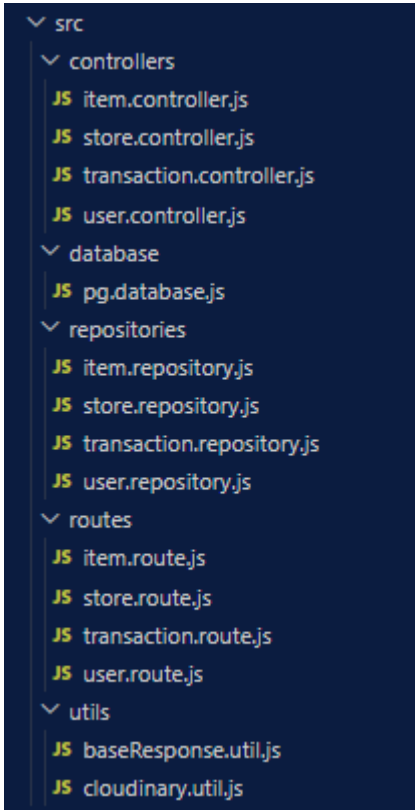
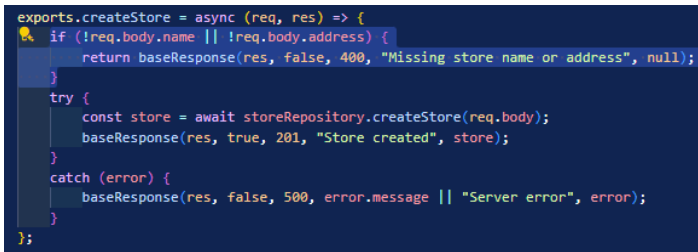
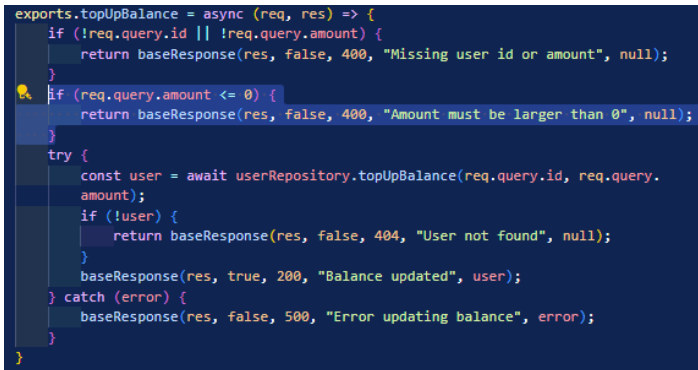
Security	<pre>const passRegex = /^(?=.*[^\s]{8,})(?=.*\d)(?=.*[^\w\d]).+\$/; exports.registerUser = async (req, res) => { if (!req.query.name !req.query.email !req.query.password) { return baseResponse(res, false, 400, "Missing user name, email, or password", null); } try { if (!emailRegex.test(req.query.email)) { return baseResponse(res, false, 400, "Invalid email", null); } if (await userRepository.getUserByEmail(req.query.email)) { return baseResponse(res, false, 400, "Email already registered", null); } if (!passRegex.test(req.query.password)) { return baseResponse(res, false, 400, "Password must be at least 8 characters long, contain at least one number, and one special character", null); } const hashedPassword = await bcrypt.hash(req.query.password, saltRounds); const user = await userRepository.registerUser(req.query.name, req.query.email, hashedPassword); baseResponse(res, true, 201, "User created", user); } catch (error) { baseResponse(res, false, 500, error.message "Server error", error); } }</pre>	Pengecekan password menggunakan passRegex saat register user dan update user, dikarenakan kemungkinan terjadinya penggunaan password yang terlalu mudah untuk dibobol.
Security	<pre>const saltRounds = 10; exports.registerUser = async (req, res) => { if (!req.query.name !req.query.email !req.query.password) { return baseResponse(res, false, 400, "Missing user name, email, or password", null); } try { if (!emailRegex.test(req.query.email)) { return baseResponse(res, false, 400, "Invalid email", null); } if (await userRepository.getUserByEmail(req.query.email)) { return baseResponse(res, false, 400, "Email already registered", null); } if (!passRegex.test(req.query.password)) { return baseResponse(res, false, 400, "Password must be at least 8 characters long, contain at least one number, and one special character", null); } const hashedPassword = await bcrypt.hash(req.query.password, saltRounds); const user = await userRepository.registerUser(req.query.name, req.query.email, hashedPassword); baseResponse(res, true, 201, "User created", user); } catch (error) { baseResponse(res, false, 500, error.message "Server error", error); } } exports.loginUser = async (email, password) => { try { const result = await db.query("SELECT * FROM users WHERE email = ?", [email]); const compare = await bcrypt.compare(password, result.rows[0].password); if (!compare) { return null; } else { return result.rows[0]; } } catch (error) { console.error("User repository error", error); } }</pre>	Penerapan hashing dengan metode bcrypt pada password agar password dapat tersimpan dalam bentuk terenkripsi dan hanya dapat didekripsi kembali menggunakan compare saat login user.
Security	<pre>const app = express(); const port = process.env.PORT 3000; const corsOptions = { origin: "https://os.netlabdte.com", methods: ["GET", "POST", "PUT", "DELETE"], }; app.use(cors(corsOptions)); app.use(express.json());</pre>	Penggunaan middleware CORS untuk mengontrol asal request agar hanya bentuk request tertentu dari domain asal tertentu yang dapat masuk ke dalam program.

Scalability

```
src > utils > JS cloudinary.util.js > ...
1  require("dotenv").config();
2  const cloudinary = require("cloudinary").v2;
3
4  cloudinary.config({
5    cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
6    api_key: process.env.CLOUDINARY_API_KEY,
7    api_secret: process.env.CLOUDINARY_API_SECRET
8  });
9
10 const uploadImage = async (image, folder) => {
11   return new Promise((resolve, reject) => {
12     if (!image) {
13       return resolve(null);
14     }
15     const base64 = Buffer.from(image.buffer).toString("base64");
16     const url = `data:${image.mimetype};base64,${base64}`;
17     cloudinary.uploader.upload(url, { folder }, (error, result) => {
18       if (error) {
19         return reject(error);
20       }
21       else {
22         return resolve(result.secure_url);
23       }
24     });
25   });
26 }
27
28 module.exports = {
29   uploadImage,
30   ...cloudinary
31 };

exports.createItem = async (req, res) => {
  const store = await storeRepository.getStoreById(req.body.store_id);
  if (!req.body.name || !req.body.price || !req.body.store_id || !req.file || !req.body.stock) {
    return baseResponse(res, false, 400, "Missing item name, price, store_id, image, or stock", null);
  }
  else if (!store) {
    return baseResponse(res, false, 404, "Store doesn't exist", null);
  }
  try {
    let image_url = null;
    if (req.file) {
      image_url = await cloudinary.uploadImage(req.file, "items");
    }
    const itemData = {
      name: req.body.name,
      price: req.body.price,
      store_id: req.body.store_id,
      stock: req.body.stock,
      image_url: image_url
    };
    const item = await itemRepository.createItem(itemData);
    baseResponse(res, true, 201, "Item created", item);
  } catch (error) {
    console.log(error);
    baseResponse(res, false, 500, error.message || "Server error", error);
  }
}
```

Integrasi dengan API Cloudinary untuk mengunggah gambar untuk item yang dibuat dan menyimpannya dalam bentuk URL ke dalam database.

Scalability		Program dipisahkan menjadi banyak bagian kecil-kecil (repository, controller, route, dan util) untuk memudahkan ekspansi dan troubleshooting pada program.
Error Handling		Memastikan body tidak kosong saat pembuatan atau pembaruan data agar tidak adanya data krusial yang bernilai NULL pada database.
Error Handling		Memastikan bahwa nilai balance pada user tidak bernilai negatif.

Error Handling	<pre> exports.createTransaction = async (req, res) => { const item = await itemRepository.getItemById(req.body.item_id); const user = await userRepository.getUserById(req.body.user_id); if (!req.body.user_id !req.body.item_id !req.body.quantity !user !item) { return baseResponse(res, false, 400, "Missing user_id, item_id, or quantity", null); } else if (req.body.quantity < 1) { return baseResponse(res, false, 400, "Quantity must be larger than 0", null); } try { const total = item.price * req.body.quantity; const transactionData = { user_id: req.body.user_id, item_id: req.body.item_id, quantity: req.body.quantity, total: total }; const transaction = await transactionRepository.createTransaction(transactionData); baseResponse(res, true, 201, "Transaction created", transaction); } catch (error) { console.error(error); baseResponse(res, false, 500, error.message "Server error", error); } } </pre>	Memastikan jumlah barang yang ingin dibeli bukan merupakan bilangan negatif sebelum membuat transaksi.
Error Handling	<pre> exports.handlePayment = async (req, res) => { const id = req.params; console.log(id); try { const transaction = await transactionRepository.getTransactionById(id); if (!transaction) { return baseResponse(res, false, 404, "Transaction not found", null); } else if (transaction.status === "paid") { return baseResponse(res, false, 400, "Transaction already paid", null); } const user = await userRepository.getUserById(transaction.user_id); if (user.balance < transaction.total) { return baseResponse(res, false, 400, "Insufficient balance", null); } const item = await itemRepository.getItemById(transaction.item_id); if (item.stock < transaction.quantity) { return baseResponse(res, false, 400, "Insufficient stock", null); } await userRepository.updateUser(user.name, user.email, user.password, user.balance - transaction.total, user.id); await itemRepository.updateItem(item.name, item.price, item.store_id, item.image_url, item.stock - transaction.quantity, item.id); const updatedTransaction = await transactionRepository.handlePayment(id); baseResponse(res, true, 200, "Payment successful", updatedTransaction); } catch (error) { console.error(error); baseResponse(res, false, 500, error.message "Server error", error); } } </pre>	Memastikan transaksi belum terbayarkan, nominal mencukupi, dan stok barang mencukupi sebelum melakukan pembayaran.
Error Handling	<pre> exports.deleteUser = async (req, res) => { try { const deleted = await userRepository.deleteUser(req.params.id); if (!deleted) { return baseResponse(res, false, 404, "User not found", null); } baseResponse(res, true, 200, "User deleted", deleted); } catch (error) { baseResponse(res, false, 500, "Error deleting user", error); } } </pre>	Memastikan bahwa objek yang terlibat ada sebelum diperbarui atau dihapus.